ECE551: Digital System Design and Synthesis

Fall 2026

Blocking vs Non-blocking Assignment

Stratified Event Queue

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Blocking vs Non-blocking Assignments

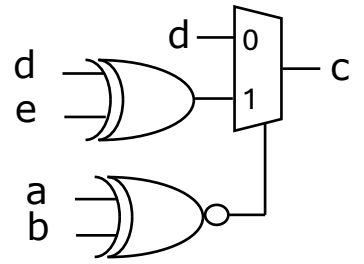
- Behavioral Verilog can describe both combinational and sequential logic

- Combinational
  - Level-triggered
  - All “inputs” (RHS nets/variables) are triggers
  - Does not depend on clock

```
reg q;

always_comb
  if (a==b) c = d ^ e;
  else c = d;
```

blocking assignment

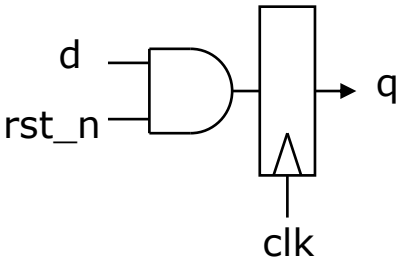


- Sequential
  - Edge-triggered by clock signal
  - Only clock (and reset) appear in trigger list
  - Can include combinational logic that feeds a FF or register

```
reg q;

always_ff @(posedge clk)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
```

non-blocking assignment



Blocking vs Non-blocking Assignments

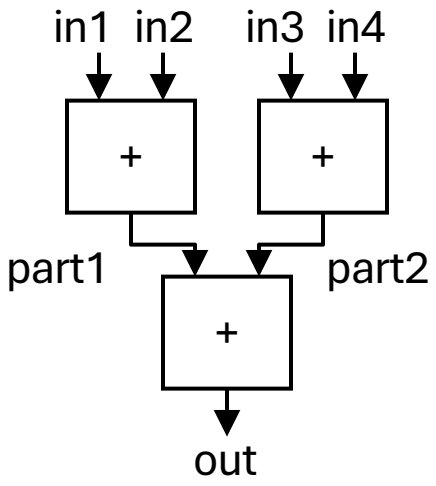
- Blocking assignment
  - Will “block” evaluation of future statements until the assignment is finished

RHS

to LHS

```
module addtree(output reg [9:0] out,
               input [7:0] in1, in2, in3, in4);

  reg [8:0] part1, part2;
  always @(in1, in2, in3, in4) begin
    part1 = in1 + in2;
    part2 = in3 + in4;
    out = part1 + part2;
  end
endmodule
```



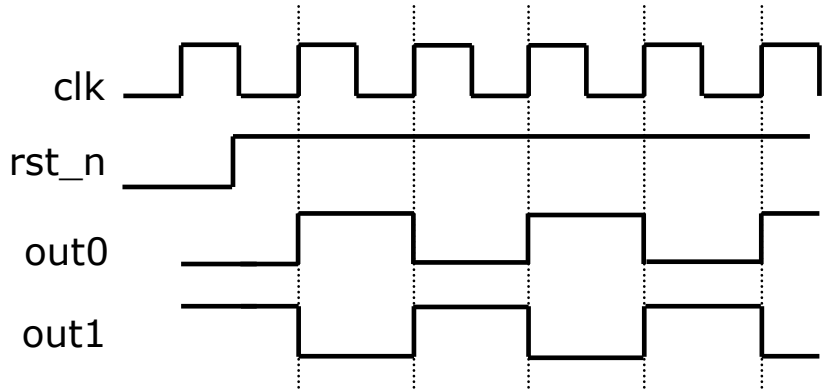
Blocking vs Non-blocking Assignments

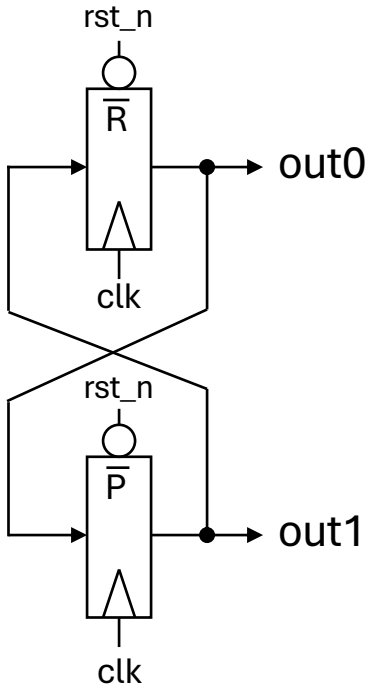
- Non-blocking assignment RHS
  - **Does not** block evaluation of future statements
    - Statements are concurrently evaluated if no delays given
  - Assignment happens at the end of the time step

to LHS

```
module swap(output reg out0, out1, input rst, clk);

  always @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
      out0 <= 1'b0;
      out1 <= 1'b1;
    end
    else begin
      out0 <= out1;
      out1 <= out0;
    end
  end
endmodule
```





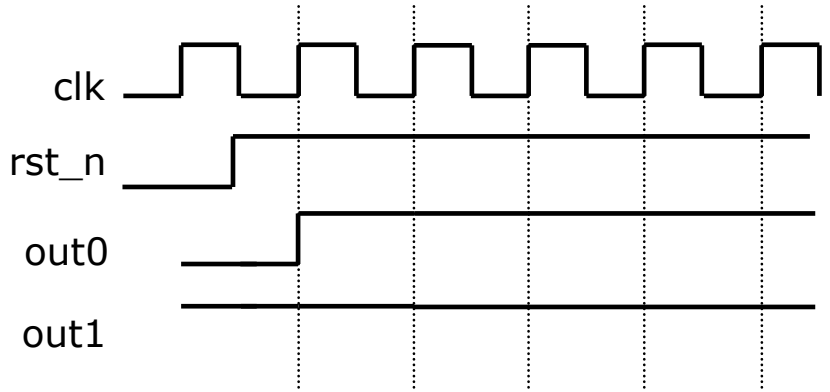
Blocking vs Non-blocking Assignments

- Non-blocking assignment RHS
  - **Does not** block evaluation of future statements
    - Statements are concurrently evaluated if no delays given
  - Assignment happens at the end of the time step

to LHS

```
module swap(output reg out0, out1, input rst, clk);

  always @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
      out0 = 1'b0;
      out1 = 1'b1;
    end
    else begin
      out0 = out1;
      out1 = out0;
    end
  end
endmodule
```



Swapping With Blocking Assignment?

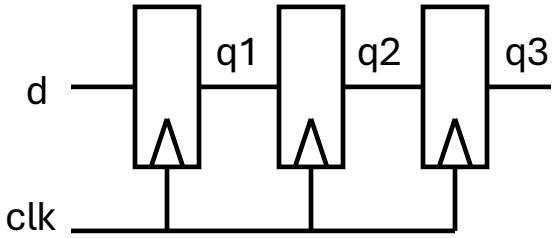
- If we really want to use blocking assignment… we need a temp register
- But don’t do this! Use non-blocking assignment.

```
module swap(output reg out0, out1, input rst, clk);

  always @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
      out0 = 1’b0;
      out1 = 1’b1;
    end
    else begin
      temp = out0;
      out0 = out1;
      out1 = temp;
    end
  end
endmodule
```

Blocking vs Non-Blocking Example

- Let's code this:



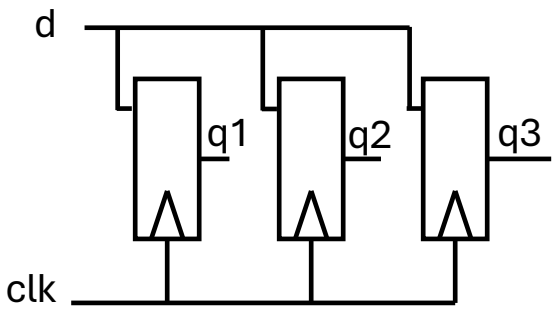
If we use blocking…

```
module pipe(clk, d, q);

  input clk,d;
  output q;
  reg q;

  always @(posedge clk) begin
    q1 = d;
    q2 = q1;
    q3 = q2;
  end

endmodule
```



If we use non-blocking…

```
module pipe(clk, d, q);

  input clk,d;
  output q;
  reg q;

  always @(posedge clk) begin
    q1 <= d;
    q2 <= q1;
    q3 <= q2;
  end

endmodule
```

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Blocking vs Non-Blocking Example

- Should we always use non-blocking then?
  - VHDL uses all non-blocking assignments, even for combinational logic
  - We could… but it is non-optimal

```
module ao4(z,a,b,c,d);

  input a,b,c,d;
  output z;

  reg z,tmp1,tmp2;

  always @(a,b,c,d) begin
    tmp1 <= a & b;
    tmp2 <= c & d;
    z <= tmp1 | tmp2;
  end
endmodule
```

Does this work?

Put tmp1 & tmp2 in the trigger list

```
module ao4(z,a,b,c,d);

  input a,b,c,d;
  output z;

  reg z,tmp1,tmp2;

  always @(a,b,c,d,tmp1,tmp2) begin
    tmp1 <= a & b;
    tmp2 <= c & d;
    z <= tmp1 | tmp2;
  end
endmodule
```

Blocking vs Non-blocking Assignments

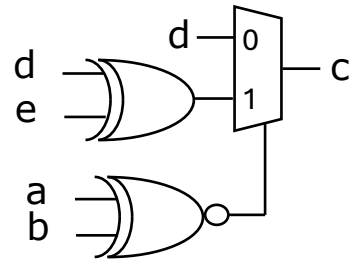
- Behavioral Verilog can describe both combinational and sequential logic

- Combinational
  - Level-triggered
  - All “inputs” (RHS nets/variables) are triggers
  - Does not depend on clock

```
reg q;

always_comb
  if (a==b) c = d ^ e;
  else c = d;
```

blocking assignment

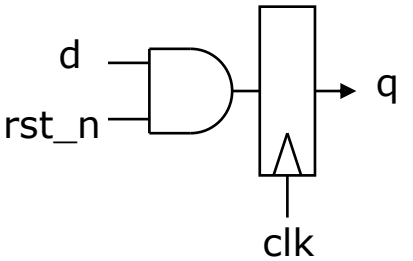


- Sequential
  - Edge-triggered by clock signal
  - Only clock (and reset) appear in trigger list
  - Can include combinational logic that feeds a FF or register

```
reg q;

always_ff @(posedge clk)
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
```

non-blocking assignment



Simulation Behavior vs Hardware Behavior

- Need to make sure simulation matches hardware
- Verilog defines how ordering of statements is interpreted by both simulator and synthesis tools
- Simulation matches hardware **if** code is well-written
  - Can have some differences with “bad” code
    - Simulator is sequential
    - Hardware is parallel
    - Race condition can occur
- Standard guarantees some scheduling order
  - Statements in the same **begin-end** block are “executed” in the order they appear
  - Statements in different **begin-end** block in same simulation time have no order guarantee

“Making it work” in **simulation** does not mean it will work after **synthesis**

Simulation Behavior vs Hardware Behavior

- Race condition – which assignment to f will occur first vs last in simulation is not known

```
module race(output reg f, input b, c);

  always @(*) begin
     f = b & c;
  end

  always @(*) begin
     f = b | c;
  end

endmodule
```

In hardware, this actually models a **short circuit**

Verilog Stratified Event Queue

- Impose a defined order on how events get processed in the same simulation step
- Terminologies:
  - Processes:
    - Objects that can be evaluated
    - Includes primitives, modules, initial and always blocks, continuous assignments, tasks, procedural assignments.
  - Update event
    - Change in the value of a net or register (LHS assignment)
  - Evaluation event
    - Computing the RHS of a statement
  - Scheduling an event
    - Putting an event on the event queue

Verilog Stratified Event Queue

- Terminologies:
  - Simulation time:
    - Time value used by simulator to model actual time
  - Simulation cycle
    - Complete processing of all current active events
  - Can be multiple simulation cycles per simulation time
  - Explicit zero delay (#0)
    - Forces process to be an inactive event instead of active
    - Incorrectly used to avoid race conditions
    - #0 does not synthesize!
    - Don’t use it

Read more in the Cummings SNUG paper

Verilog Stratified Event Queue

- Events can go on one of 5 queues
- Region 1: Active Events
  - Most events except those explicitly in other regions
  - Includes $display system tasks
- Region 2: Inactive Events
  - Processed after all active events
  - #0 delay events (bad!)
- Region 3: Non-blocking Assign Update Events
  - Evaluation previously performed
  - Update is after all active and inactive events complete
- Region 4: Monitor Events
  - Caused by $monitor and $strobe system tasks
- Region 5: Future Events
  - Occurs at some future simulation time
  - Includes future events of other regions
  - Other regions only contain events for CURRENT simulation time

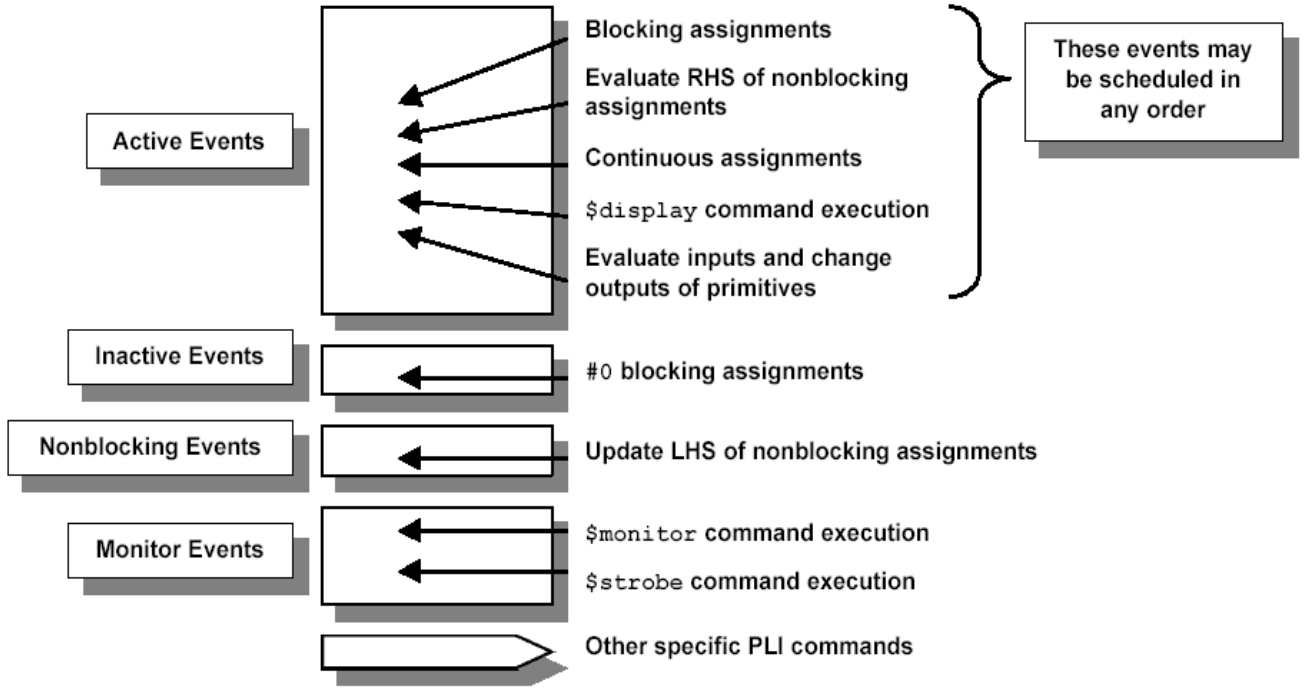


Figure 1 - Verilog "stratified event queue"

Simulation Model

```
Let T be current simulation time
while (there are events) {
  if (no active events) {
        if (inactive events) activate all inactive events
        else if (n.b. update events) activate all n.b. update events
        else if (monitor events) activate all monitor events
        else {   advance T to the next event time
                 activate all future events for time T }
  }
  E = pull from active event queue
  if (E is an update event) {
        update the modified object
        add evaluation events for sensitive processes to the event queue
  } else { // evaluation event
        evaluate the process
        add update event(s) to the event queue
  }
}
```

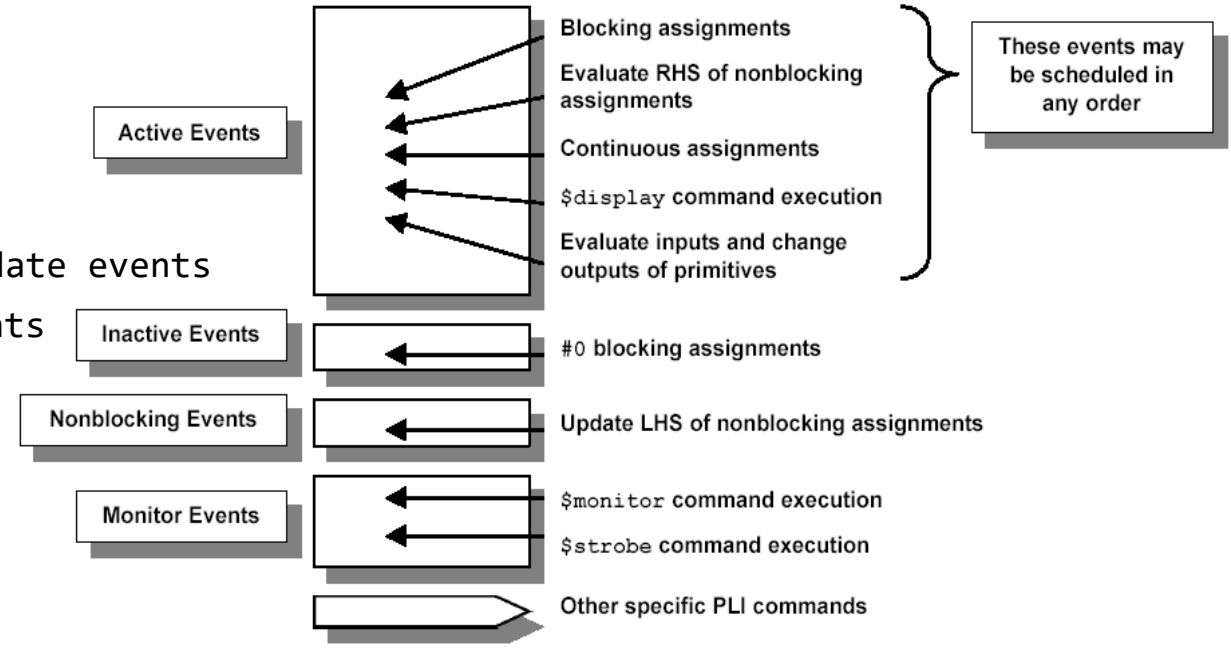


Figure 1 - Verilog "stratified event queue"

Stratified Event Queue Example

- What is displayed?

```
assign p = q;

initial begin
  q = 1;
  #1 q = 0;
  $display(p);
end
```

Hand Simulation

Active:	NB Update:	Future:
~~p=x~~		~~#1 q = 0~~
~~q=1~~		~~$display(p)~~
~~p=1~~		
~~q = 0~~		
~~$display(p)~~		
~~p = 0~~		

Time	Event/Action
0	p = x
0	q = 1
0	p = 1
1	q = 0
1	$display(1)
1	p = 0

- What if $strobe(p) was used instead?

Stratified Event Queue Example

```
always @(posedge clk, negedge rst_n)
  if (!rst_n)
    FF1 <= 1'b0;
  else
    FF1 <= d;

always @(posedge clk)
  FF2 <= FF1;

initial begin
  clk = 0;
  rst_n = 0;
  d = 1;
  #5 rst_n = 1;
  #5 clk = 1;
end
```

Absolutely initially:

FF1 = x clk = x d=x

FF2 = x rst_n=x

Active:	NB Update:	Future:
~~clk = 0~~	~~FF1 <= 0~~	~~#5 rst_n = 1~~
~~rst_n = 0~~	~~FF1 <= 1~~	~~#5 clk = 1~~
~~d = 1~~	~~FF2 <= 0~~	
~~always_FF1~~		
~~FF1 = 0~~		
~~rst_n = 1~~		
~~clk = 1~~		
~~always_FF1~~		
~~always_FF2~~		
~~FF1 = 1~~		
~~FF2 = 0~~		

Time	Event/Action
0	clk = 0
0	rst_n = 0
0	d = 1
0	FF1 = 0
5	rst_n = 1
10	clk = 1
10	FF1 = 1
10	FF2 = 0

Verilog Stratified Event Queue

- Should understand the order statements are evaluated
- Active vs non-blocking vs future event regions
- $display vs $strobe vs $monitor
- Separation of evaluation and update of non-blocking

- Read the Cummings SNUG paper posted on the course website
  - Well written easy to understand paper
  - Outlines 8 guidelines for good Verilog coding
    - Learn them
    - Use them